Table of Contents

1. Introduction

2. Object Detection Fundamentals

3. YOLO Overview

4. SSD Overview

5. Faster R-CNN Overview

6. Comparative Analysis

7. Suitability for Real-Time Systems

8. Selection for the Project

9. Conclusion

References

1. Introduction

Module 2 Option 5 requires a comparative study of YOLO, SSD, and Faster R-CNN with attention to accuracy, speed, and suitability for real-time operation, followed by a project focused on real-time object detection using YOLO. This report addresses the research component and provides the technical basis for the implementation.

Object detection combines object classification with spatial localization. A detector therefore needs to answer two questions for each object: what class is present and where it is located in the image. In an autonomous-vehicle context, this capability is part of visual perception and can support downstream functions such as scene understanding, tracking, and decision-making.

2. Object Detection Fundamentals

- Classification assigns a class label to an image or region.
- Localization estimates an object bounding box.
- Object detection performs both tasks for multiple objects in a scene.
- A practical real-time detector must balance detection quality with computational latency.

Common output fields include a bounding box, a class identifier, and a confidence score. The project implementation stores these frame-by-frame and adds timing measurements so that detection quality and runtime behavior can be studied together.

3. YOLO Overview

YOLO (You Only Look Once) introduced a single-network formulation in which object locations and class probabilities are predicted in one evaluation of the image. The original work emphasized end-to-end detection and real-time processing. The paper reported 45 frames per second for its base model and 155 frames per second for Fast YOLO on the hardware and implementation used in that study; these historical figures should not be treated as performance guarantees for current models.

For this project, the implementation uses the Ultralytics Python interface. The current official documentation provides Python and CLI prediction flows for image, video, and webcam sources. The model checkpoint is configurable; the default project configuration is yolo26n.pt, consistent with the current Ultralytics quickstart documentation.

4. SSD Overview

SSD (Single Shot MultiBox Detector) is also a single-stage detector. It predicts class scores and bounding-box adjustments from multiple feature maps with different resolutions. The original SSD paper reports 72.1% mAP at 58 FPS for a 300x300 model on a Nvidia Titan X for its VOC2007 experiment. Again, the result is historical and hardware-specific.

5. Faster R-CNN Overview

Faster R-CNN is a two-stage detection architecture. A Region Proposal Network first generates candidate object regions, and a second stage performs detection using shared convolutional features. The original paper reported about 5 FPS for a VGG-16 system including all steps on the GPU used in that study, illustrating the higher computational cost of the two-stage approach.

6. Comparative Analysis

Criterion	YOLO	SSD	Faster R-CNN
Detection style	Single-stage / end-to-end	Single-stage / multi-scale	Two-stage / region proposals
Core idea	Predict boxes and classes in one network evaluation	Predict boxes/classes from multiple feature maps	Generate proposals then classify/refine them
Real-time suitability	Strong focus on low-latency inference	Strong real-time orientation	Higher compute cost; historically slower
Implementation complexity	Moderate with modern libraries	Moderate	Higher
Small-object handling	Depends on model/version and input size	Multi-scale feature maps help	Strong proposal-based localization, but computationally heavier
Project fit	Direct match to Option 5 implementation	Reference comparator	Reference comparator

The table is a conceptual comparison. Exact accuracy and speed depend on architecture version, checkpoint, dataset, hardware, input size, preprocessing, and runtime configuration.

7. Suitability for Real-Time Systems

For a real-time perception pipeline, latency matters because detections arrive continuously from a camera stream. A detector with high per-frame latency can reduce effective frame rate and increase the age of information passed to downstream modules. The project therefore measures inference time and frame rate instead of reporting a single generic “real-time” label.

- Latency should be measured on the actual target hardware.
- Input resolution should be recorded because it affects computational cost.
- Confidence threshold should be recorded because it changes the set of retained detections.
- Hardware acceleration, when available, should be documented alongside results.

8. Selection for the Project

YOLO is selected for the implementation because the Module 2 project statement explicitly asks for "Analysis of Real-Time Object Detection Using YOLO." The developed system adds a performance-analysis layer around inference so the submission demonstrates not only object detection but also measurement and reproducibility.

9. Conclusion

YOLO, SSD, and Faster R-CNN represent different points in the accuracy-computation-design space. The original papers established the basic trade-offs between single-stage and two-stage detection. For the Module 2 project, YOLO provides a direct implementation path while still allowing rigorous analysis through measured latency, FPS, detection counts, and confidence statistics.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," arXiv:1506.02640, 2015; published at CVPR 2016. https://arxiv.org/abs/1506.02640

[2] W. Liu et al., "SSD: Single Shot MultiBox Detector," arXiv:1512.02325, 2015; published at ECCV 2016. https://arxiv.org/abs/1512.02325

[3] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," arXiv:1506.01497, 2015; published at NeurIPS 2015. https://arxiv.org/abs/1506.01497

[4] Ultralytics, "Install Ultralytics," and "Modes / Predict," official documentation, accessed 28 Sep 2026. https://docs.ultralytics.com/quickstart and https://docs.ultralytics.com/modes